

Phase-Based Container Security Framework: Adaptive Syscall Filtering and AI-Driven Threat Detection

Guide:

Dr. S . Bose

Professor, DCSE

Department of Computer Science and Engineering

Bharath D - 2025165027

**M.E Computer Science and Engineering
(Data Science and Cybersecurity)**

INTRODUCTION

- **Container security** has become critical as containerized deployments share the host kernel, making the syscall interface the primary attack surface for exploits and privilege escalation.
- Default container runtimes (e.g., Docker's default Seccomp profile) permit **300+ syscalls** by default, far more than most workloads genuinely need — leaving substantial unnecessary attack surface.
- Static allowlisting alone cannot stop attacks that abuse an **already-permitted syscall** through malicious arguments or sequences, once a container is compromised.
- A **eBPF-based** framework is proposed to progressively restrict permitted syscalls and continuously monitor runtime behavior against them.

OVERALL OBJECTIVE

Phase 1: To ensure attack surface reduction by developing a *static-and-dynamic seccomp profiling* pipeline — combining binary/call-graph analysis with eBPF-based runtime monitoring and an adaptive, change-point-driven phase segmentation mechanism — that restricts containers to only the syscalls they genuinely require.

Phase 2: To utilize eBPF-based sliding-window syscall capture and a hybrid *VAE + Isolation Forest* anomaly model with *argument-level context*, ensuring real-time behavioral integrity of container syscall activity and triggering risk-based alerting or blocking on detected anomalies.

LITERATURE SURVEY

S.No	Publication Details	Methodologies Used	Limitations
1.	* Enhancing Container Security Through Phase-Based System Call Filtering (PSF) <i>K. Chen et al.</i> <i>IEEE Trans. Cloud Computing, 2025</i>	- Static + dynamic eBPF profiling → S-SF, D-SF, I-SF - Cosine-similarity segmentation (0.8, 5s) → R-SF - Redis: 134→69 C; validated on 10 real images	- Fixed similarity threshold, not adaptive to workload - frequency-only features are order-blind
2.	DeSFAM: An Adaptive eBPF and AI-Driven Framework for Securing Cloud Containers in Real Time <i>S. Zehra et al.</i> <i>IEEE Access, 2025</i>	- Hybrid static-dynamic profiling; SyscallAD (VAE + iForest) - MITRE ATT&CK/CVE-based risk scoring; eBPF+LSM enforcement 94% precision, 90% recall, sub-ms latency, <1% overhead	No argument-level syscall analysis (self-identified)
3.	Detecting Cryptojacking Containers Using eBPF-Based Security Runtime and ML <i>R. Kim et al.</i> <i>Electronics, 2025</i>	- eBPF/Tetragon tracing - N-gram features + ML classifiers - Up to 99.75% classification accuracy	N-gram features may not generalize to novel/unseen mining techniques

S.No	Publication Details	Methodologies Used	Limitations
4.	FedMon: Federated eBPF Monitoring for Distributed Anomaly Detection <i>S. Zehra et al.</i> <i>Proc. ICET, 2025</i>	• Per-cluster eBPF agents + federated learning • Shared VAE + iForest model updates only • 94% precision, 91% recall; 60% bandwidth reduction	Cross-cluster heterogeneity can affect model convergence
5.	An In-Depth Analysis of eBPF-Based System Security Tools <i>J. Her et al.</i> <i>IEEE Access, 2025</i>	• Comparative eval: KubeArmor, Falco, Tetragon, Tracee • Hook-point, policy-scope, overhead analysis	Comparative survey only — no new detection mechanism proposed
6.	BeaCon: Automatic Container Policy Generation Using Environment-Aware Dynamic Analysis <i>H. Kang et al.</i> <i>Computers & Security, 2026</i>	• eBPF monitor for syscalls/capabilities • Emulated environment exploration + scoring • Low-overhead, mitigates realistic kernel attacks	Effectiveness depends on quality of emulated exploration; may miss rarely triggered paths

S.No	Publication Details	Methodologies Used	Limitations
7.	An Abnormal File Access Detection Model for Containers Based on eBPF Listening <i>N. Zhou et al.</i> <i>Mathematics, 2026</i>	• Lightweight eBPF file-access monitoring • Multimodal path-semantic features • Real-time, fine-grained detection	Scope limited to file-access-based attack precursors only
8.	Hybrid Runtime Detection of Malicious Containers Using eBPF <i>J. Ryu et al.</i> <i>Computers, Materials & Continua, 2026</i>	• eBPF flow + syscall traces • Multi-class ML classification • 98.39% host-based accuracy	Evaluated on a limited set of malware/benign scenarios
9.	Unsupervised Real-Time In-Kernel Intrusion Detection Using Autoencoders and eBPF <i>H. Taguchi et al.</i> <i>IEICE Trans. Communications, 2026</i>	• Unsupervised autoencoder, quantization-aware training • Fixed-point in-kernel inference via BPF maps • ~70× throughput gain vs. user-space IDS	eBPF stack/instruction-count limits constrain achievable model size
10	eBPF-Guard: A Detection Method for Container Escape via Multi-Level Monitoring <i>X. Lin et al.</i> <i>Empirical Software Engineering, 2026</i>	• Multi-level eBPF probes (syscalls + call features + logs) • LLM-based analysis stage • Surfaces unseen escape indicators	Runtime LLM inference adds latency/resource cost

Summary of Issues

1. Shared kernel exposes a large attack surface
2. Default Seccomp configurations are overly permissive
3. Manual Seccomp policy authoring is impractical
4. Existing automated approaches don't account for execution phase

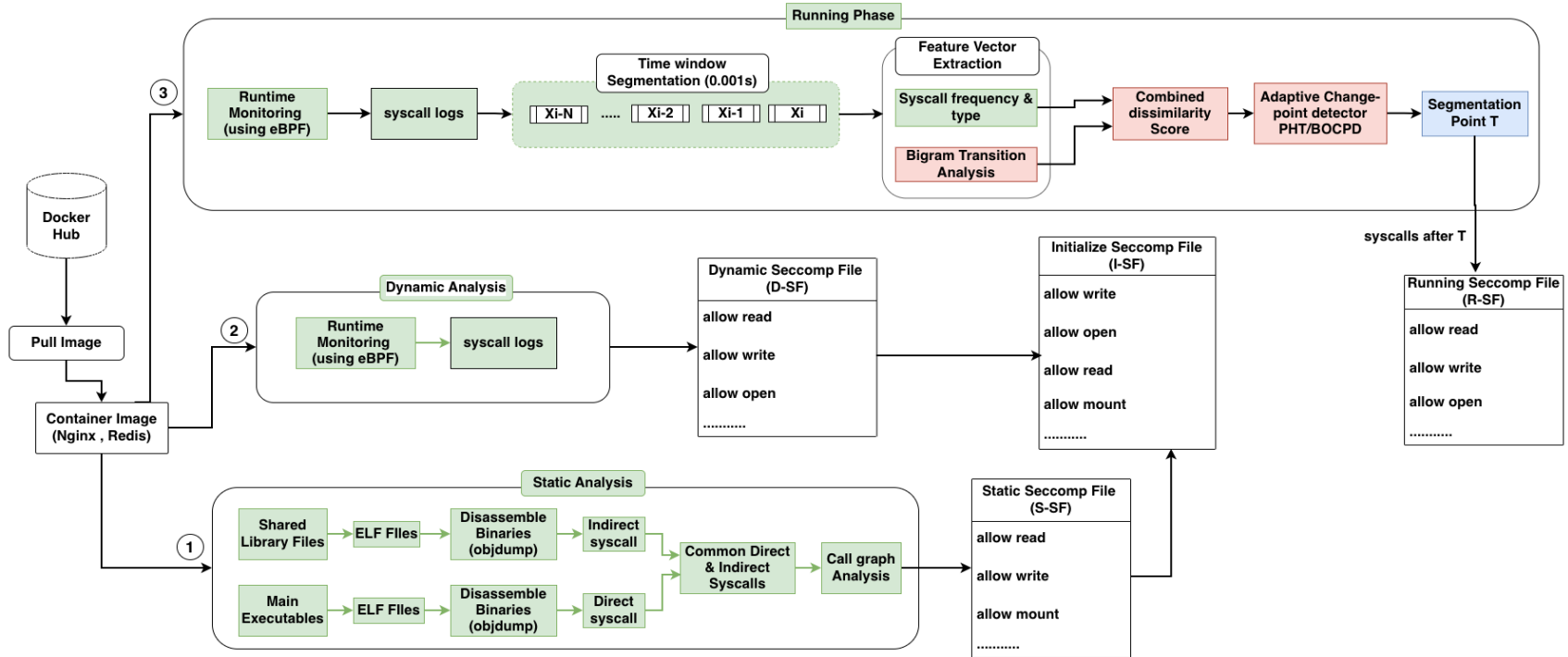
PROPOSED SYSTEM – PHASE 1

- **Static Analysis:** Disassembles shared libraries and main executables (ELF, objdump) to extract direct and indirect syscalls via call graph analysis, producing the Static Seccomp File (S-SF).
- **Dynamic Analysis:** Uses eBPF-based runtime monitoring to capture syscall logs during execution, producing the Dynamic Seccomp File (D-SF). S-SF and D-SF are merged into the Initialize Seccomp File (I-SF).
- **Adaptive Running-Phase Segmentation:** Extracts syscall frequency and bigram transition features per time window, combines them into a dissimilarity score, and applies change-point detection (Page-Hinkley/BOCPD) to identify segmentation point T — replacing fixed-threshold detection with a workload-calibrated mechanism.
- **Running Seccomp File Generation:** Syscalls observed after T are compiled into the Running Seccomp File (R-SF), the enforced allowlist for steady-state operation.

PROPOSED SYSTEM - PHASE 2

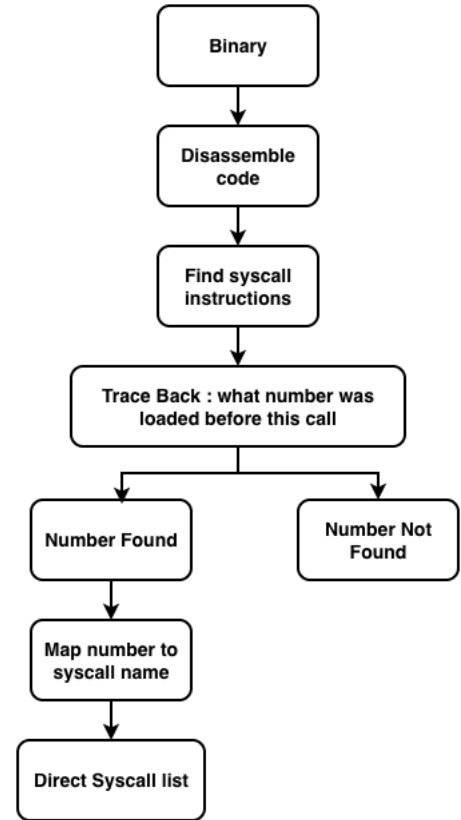
- **eBPF-Based Syscall Capture:** Continuously captures live syscalls from running containers into a sliding window buffer .
- **Argument-Aware Feature Extraction:** Aggregates syscall sequences per window into feature vectors that include argument-level context.
- **Hybrid Anomaly Model:** A VAE + Isolation Forest ensemble, trained offline on DongTing and BETH datasets, evaluates each window and classifies it as normal or anomalous.
- **Risk Scoring and Response:** Anomalous sequences are scored by a Risk Scoring Engine; medium-risk sequences trigger Log/Alert, high-risk sequences trigger Block.

HIGH LEVEL BLOCK DIAGRAM FOR PHASE-1



Module - 1 Direct Analysis

```
for each binary:
    instructions = disassemble(binary)
    for each instruction in instructions:
        if instruction triggers a syscall:
            number = trace_back_for_constant(instruction)
            if number found:
                name = lookup_syscall_name(number)
                record(RESOLVED, number, name)
            else:
                record(UNRESOLVED)
    return all records
```



Direct Analysis Example

```
section .data
    msg db "Hello", 10      ; The text data payload to print

section .text
    global _start

_start:
    mov rax, 1              ; 1 is the numeric ID for sys_write on x86-64 Linux
    mov rdi, 1              ; Argument 1: File descriptor (1 = stdout)
    mov rsi, msg            ; Argument 2: Memory address pointer to our text data
    mov rdx, 6              ; Argument 3: The number of bytes to print
    syscall                ; The CPU jumps to the kernel to perform the action

    mov rax, 60             ; 60 is the numeric ID for sys_exit
    mov rdi, 0              ; Argument 1: Exit status code (0 = success)
    syscall                ; The CPU jumps to the kernel to terminate the program
```

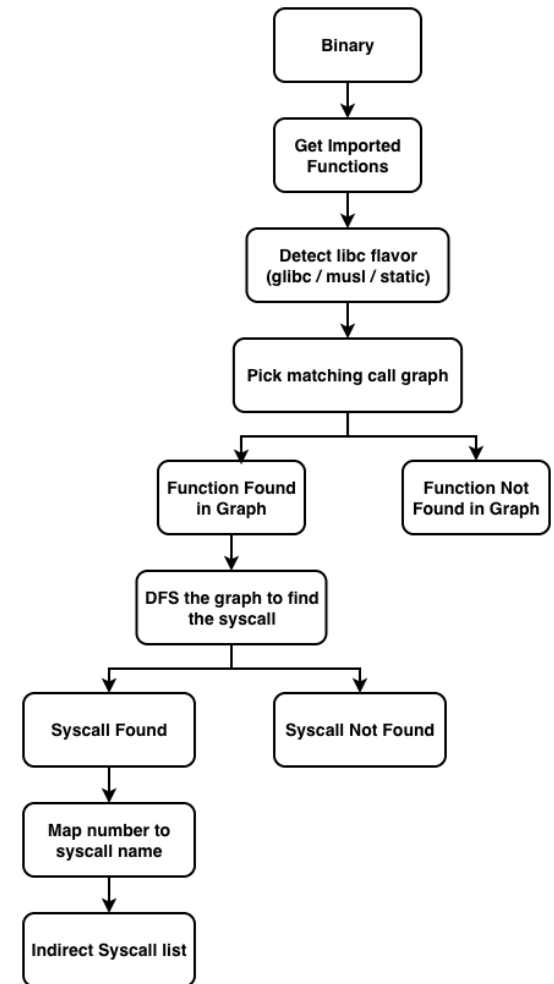
Module - 1 Indirect Analysis

```
for each binary:
    imports = get_imported_functions(binary)
    libc = detect_libc_flavor(binary)
    graph = load_callgraph_for(libc)

    for each function in imports:
        if graph has no matching node for function:
            record(UNRESOLVED, "not in graph")
            continue

        syscalls = trace_calls(graph, function) # follow calls until syscall found
        if syscalls is empty:
            record(UNRESOLVED, "no syscall reachable")
        else:
            for each syscall_number in syscalls:
                name = lookup_syscall_name(syscall_number)
                record(RESOLVED, syscall_number, name)

    return all records
```



Indirect Analysis Example

```
SECTION .data
    msg db "Hello", 10          ; The string "Hello" followed by a newline (\n)
    msg_len equ $ - msg        ; Automatically calculates string length (6 bytes)

SECTION .text
    ; Declare libc functions as external symbols so the linker can find them
    extern write
    extern exit

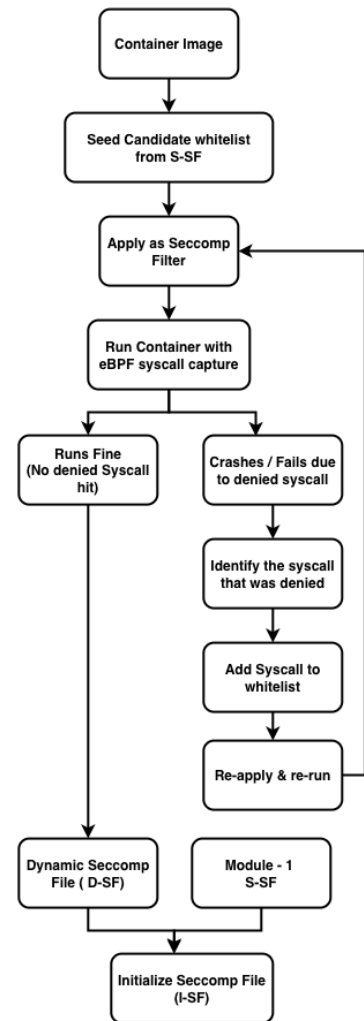
    global _start

_start:
    ; 1. Call libc's write() instead of issuing "syscall" ourselves
    ; System V AMD64 ABI placement: rdi, rsi, rdx
    mov edi, 1                  ; Argument 1: file descriptor (1 = stdout)
    mov rsi, msg                ; Argument 2: pointer to the string data
    mov rdx, msg_len            ; Argument 3: number of bytes to write (6)
    call write                  ; Control transfers to libc wrapper, which triggers the syscall

    ; 2. Call libc's exit() instead of issuing "syscall" ourselves
    mov edi, 0                  ; Argument 1: exit status code (0 = success)
    call exit                   ; Control transfers to libc termination logic
```

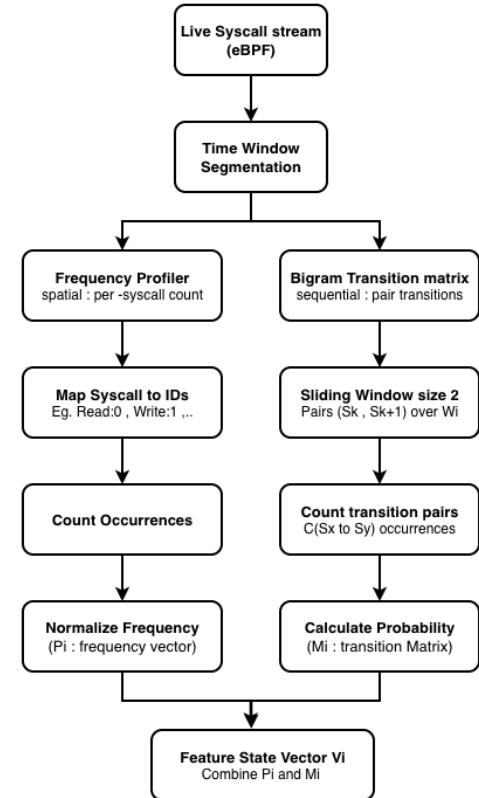
Module - 2 Dynamic Analysis

```
function dynamic_analysis(container_image, S_SF):  
    candidate_whitelist = copy(S_SF)          # seed from static analysis result  
  
    loop:  
        apply_seccomp_filter(container_image, candidate_whitelist)  
        run(container_image)                  # under eBPF monitoring  
        syscalls_seen = capture_syscalls(container_image)  
  
        if container_image crashed:  
            missing_syscall = identify_denied_syscall(crash_info)  
            candidate_whitelist.add(missing_syscall)  
            continue                          # loop back and re-run with updated whitelist  
        else:  
            break                             # ran clean - stable whitelist found  
  
    D_SF = candidate_whitelist  
    return D_SF  
  
# Later:  
I_SF = merge(S_SF, D_SF)
```



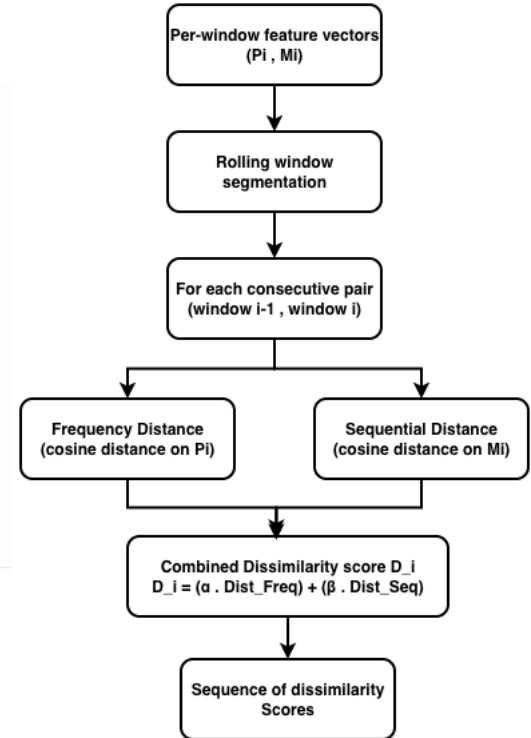
Module - 3 Running Phase (Feature Extraction)

```
function feature_vector_extraction(container_image):  
    syscalls = capture_syscalls_via_eBPF(container_image)  
    windows = time_window_segmentation(syscalls)  
  
    for each window in windows:  
        Pi = frequency_profiler(window)      # map -> count -> normalize  
        Mi = bigram_transition_matrix(window) # sliding pairs -> count ->  
probability  
        feature[window] = combine(Pi, Mi)  
  
    return feature, syscalls
```



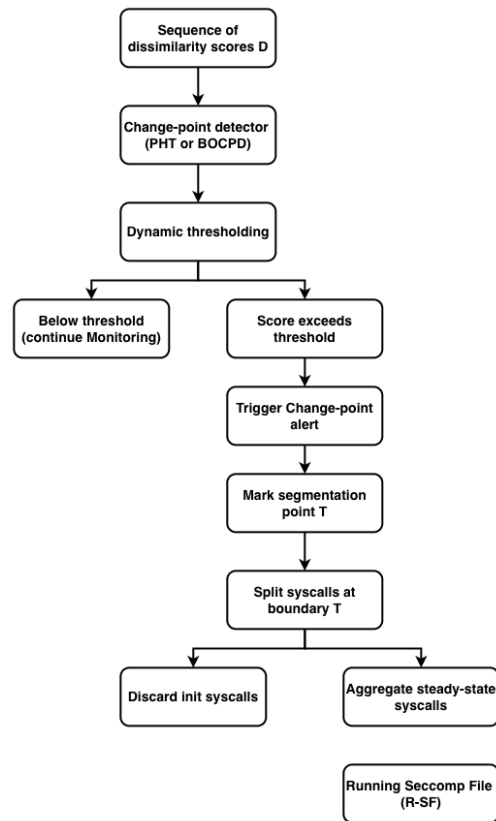
Module - 3 Running Phase (Dissimilarity Computation)

```
function dissimilarity_computation(feature):  
    P, M = split_into_frequency_and_transition(feature)  
    rolling_windows = rolling_window_segmentation(P, M)  
  
    for each consecutive pair i in rolling_windows:  
        Dist_Freq[i] = 1 - ( dot(P[i], P[i-1]) / (norm(P[i]) * norm(P[i-1])) )  
        Dist_Seq[i] = 1 - ( dot(flatten(M[i]), flatten(M[i-1])) /  
                             (norm(flatten(M[i])) * norm(flatten(M[i-1]))) )  
        D[i] = (alpha * Dist_Freq[i]) + (beta * Dist_Seq[i])  
  
    return D
```

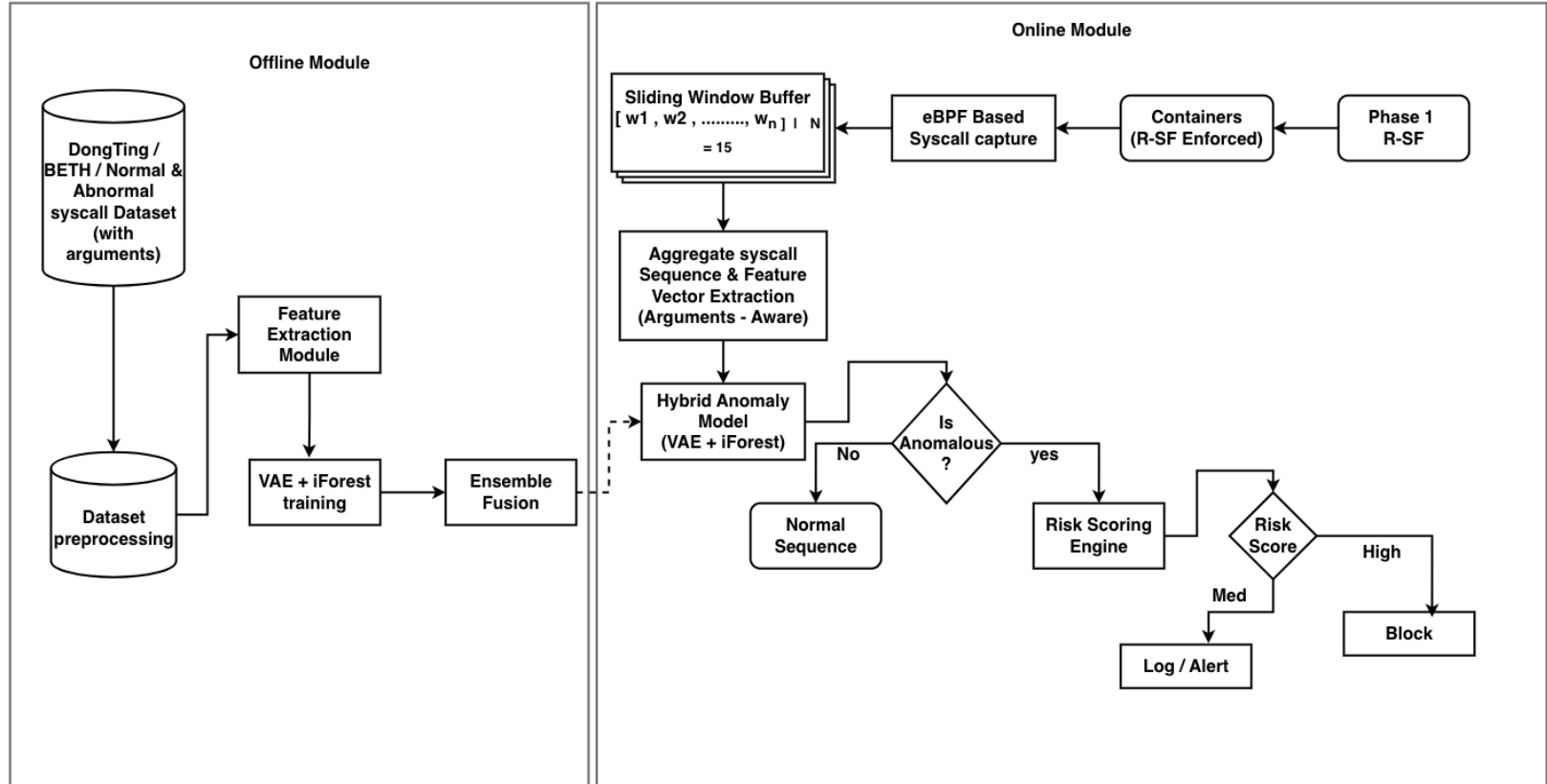


Module - 3 Running Phase (Adaptive Change-Point Detection)

```
function adaptive_change_point_detection(D, syscalls):  
    for each D[i] in D:  
        score[i] = change_point_detector(D[i])    # PHT or BOCPD, selected  
                                                    empirically  
  
    threshold = dynamic_thresholding(score)  
  
    if score exceeds threshold:  
        trigger_change_point_alert()  
        T = mark_segmentation_point()  
    else:  
        continue_monitoring()  
  
    init_syscalls, steady_state_syscalls = data_splitter(syscalls, boundary=T)  
    discard(init_syscalls)  
    R_SF = aggregate_syscalls(steady_state_syscalls)  
  
    return R_SF
```



HIGH LEVEL BLOCK DIAGRAM FOR PHASE-2



Code Snippets

```
def _classify_syscall_instruction(
    self,
    instruction,
    architecture: str,
    plt_symbols: Dict[int, str],
    func_symbols: Dict[int, str],
    got_symbols: Dict[int, str],
) -> Optional[str]:
    mnemonic = instruction.mnemonic

    # Raw machine syscall instruction
    if mnemonic == "syscall":
        return "raw_syscall"
    if mnemonic == "int":
        if (
            hasattr(instruction, "operands")
            and len(instruction.operands) == 1
            and instruction.operands[0].type == CS_OP_IMM
            and instruction.operands[0].imm == 0x80
        ):
            return "raw_int80"
        if hasattr(instruction, "op_str") and "0x80" in instruction.op_str:
            return "raw_int80"
    if mnemonic == "sysenter":
        return "raw_sysenter"
    if mnemonic in {"svc", "swi"}:
        return "raw_svc"

    # Function calls to syscall wrappers
    if mnemonic in {"call", "jmp"} and hasattr(instruction, "operands"):
        operands = instruction.operands
        if len(operands) == 1:
            op = operands[0]
            callee_name: Optional[str] = None
            if op.type == CS_OP_IMM:
                target_addr = op.imm
                callee_name = plt_symbols.get(
                    target_addr
                ) or func_symbols.get(target_addr)
            elif op.type == CS_OP_MEM:
                if architecture == "x86_64":
                    target_got = (
                        instruction.address + instruction.size + op.mem.disp
                    )
```

```
def _resolve_syscall_number(
    self,
    instructions,
    syscall_index: int,
    target_reg: str = "rax",
    architecture: str = "x86_64",
) -> ResolutionResult:
    start = max(0, syscall_index - self.backtrack_window)
    window = instructions[start:syscall_index]

    canonical_target = REG_CANONICAL.get(
        target_reg.lower(), target_reg.lower()
    )
    reg_state: Dict[str, int] = {}
    confidence_state: Dict[str, str] = {}
    stack_immediates: List[int] = []

    for insn in window:
        mnemonic = insn.mnemonic
        operands = getattr(insn, "operands", [])

        if self._is_call_or_ret(insn):
            # When checking raw syscall in rax, an unrelated call clobbers rax
            # Caller-saved registers are clobbered
            for clobbered in [
                "rax",
                "rcx",
                "rdx",
                "rsi",
                "rdi",
                "r8",
                "r9",
                "r10",
                "r11",
            ]:
                reg_state.pop(clobbered, None)
                confidence_state.pop(clobbered, None)
                continue

            if mnemonic == "push" and len(operands) == 1:
                op = operands[0]
                if op.type == CS_OP_IMM:
                    stack_immediates.append(int(op.imm))
                continue

            if mnemonic == "xor" and len(operands) == 2:
                if (
                    operands[0].type == CS_OP_REG
                    and operands[1].type == CS_OP_REG
                ):
```

Code Snippets

```
@staticmethod
def _resolve_callgraph_function_name(function: str, analyzer) -> Optional[str]:
    if analyzer is None:
        return None
    candidates = [function]
    if "@" in function:
        candidates.append(function.split("@", 1)[0])
    clean = candidates[-1]
    if clean.startswith("_") and len(clean) > 2:
        candidates.append(clean[2:])
    elif clean.startswith(".") and len(clean) > 1:
        candidates.append(clean[1:])
    else:
        candidates.append("_" + clean)
    if clean.endswith(".64"):
        candidates.append(clean[:-2])

    for cand in candidates:
        if analyzer.has_function(cand):
            return cand
    return None

def _select_analyzer(self, libc: str):
    if libc == "glibc":
        return self.glibc_analyzer if self.glibc_analyzer.available() else None
    if libc == "musl":
        return self.musl_analyzer if self.musl_analyzer.available() else None
    if self.glibc_analyzer.available():
        return self.glibc_analyzer
    if self.musl_analyzer.available():
        return self.musl_analyzer
    return None

@staticmethod
def _detect_libc(metadata: ELFMetadata) -> str:
    interpreter = (metadata.interpreter or "").lower()
    needed = [lib.lower() for lib in metadata.needed_libraries]
    if "musl" in interpreter or any("musl" in lib for lib in needed):
        return "musl"
    if "ld-linux" in interpreter or any(lib.startswith("libc.so") for lib in needed):
        return "glibc"
    if metadata.link_type == "static":
        return "static"
    return "glibc" if metadata.is_dynamic else "unknown"
```

```
@staticmethod
def parse_syscall_node(node: str) -> Optional[int]:
    match = SYSCALL_NODE_RE.match(node)
    if not match:
        return None
    return int(match.group(1))

def get_syscalls_from_function(self, function: str) -> Set[int]:
    return {item.number for item in self.get_syscalls_with_paths(function)}

def get_syscalls_with_paths(self, function: str) -> List[SyscallPath]:
    if function not in self.graph:
        return []

    found: Set[Tuple[int, Tuple[str, ...]]] = set()
    results: List[SyscallPath] = []
    visited_nodes: Set[str] = set()
    stack: List[Tuple[str, List[str]]] = [(function, [])]

    while stack:
        node, path = stack.pop()
        if node in visited_nodes:
            continue
        visited_nodes.add(node)

        syscall_num = self.parse_syscall_node(node)
        if syscall_num is not None:
            path_tuple = tuple(path + [node])
            marker = (syscall_num, path_tuple)
            if marker not in found:
                found.add(marker)
                results.append(SyscallPath(number=syscall_num, path=list(path_tuple)))
            continue

        for child in self.graph.get(node, set()):
            if child in visited_nodes:
                continue
            stack.append((child, path + [node]))

    return results

def has_function(self, function: str) -> bool:
    return function in self.graph

def nodes(self) -> Iterable[str]:
    return self.graph.keys()
```

Output

```
126 capset @ 0x5e345 (high)
207 io_destroy @ 0x5fb8b (high)
208 io_getevents @ 0x60125 (high)
206 io_setup @ 0x6044f (high)
207 io_destroy @ 0x60659 (high)
321 bpf @ 0x61494 (high)
321 bpf @ 0x6156f (high)
321 bpf @ 0x61641 (high)
321 bpf @ 0x616ce (high)
321 bpf @ 0x61769 (high)
0x61909 unresolved (low)
186 gettid @ 0x6290f (high)
```

```
330 pkey_alloc <- pthread_cond_signal | pthread_cond_signal -> syscall ( 330 )
331 pkey_free <- pthread_cond_signal | pthread_cond_signal -> syscall ( 331 )
327 preadv2 <- pthread_cond_signal | pthread_cond_signal -> syscall ( 327 )
325 mlock2 <- pthread_cond_signal | pthread_cond_signal -> syscall ( 325 )
322 execveat <- pthread_cond_signal | pthread_cond_signal -> syscall ( 322 )
318 getrandom <- pthread_cond_signal | pthread_cond_signal -> syscall ( 318 )
134 uselib <- pthread_cond_signal | pthread_cond_signal -> syscall ( 134 )
329 pkey_mprotect <- pthread_cond_signal | pthread_cond_signal -> syscall ( 329 )
15 rt_sigreturn <- pthread_cond_signal | pthread_cond_signal -> syscall ( 15 )
```

```
166 umount2 <- pthread_cond_signal | pthread_cond_signal -> __pthread_cond_signal_2_0 -> calloc -> syscall ( 166 )
326 copy_file_range <- pthread_cond_signal | pthread_cond_signal -> __pthread_cond_signal_2_0 -> calloc -> syscall ( 326 )
328 pwritev2 <- pthread_cond_signal | pthread_cond_signal -> __pthread_cond_signal_2_0 -> calloc -> syscall ( 328 )
38 setitimer <- pthread_cond_signal | pthread_cond_signal -> __pthread_cond_signal_2_0 -> calloc -> me -> setitimer -> __setitimer -> syscall ( 38 )
```

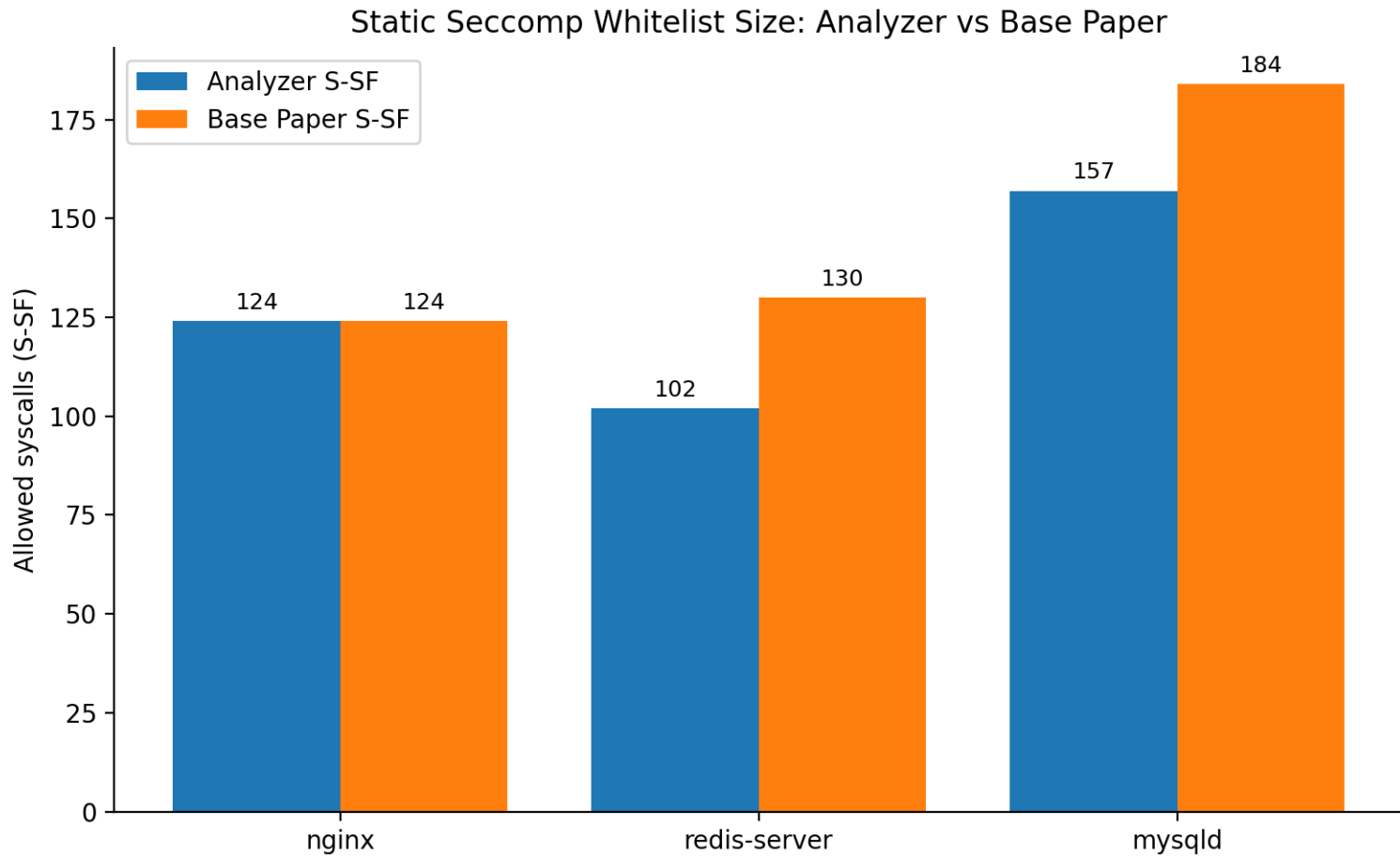
Output - S-SF for Nginx (124 syscalls total)

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": [
    "SCMP_ARCH_X86_64"
  ],
  "syscalls": [
    {
      "names": [
        "accept",
        "accept4",
        "access",
        "arch_prctl",
        "bind",
        "bpf",
        "capset",
        "chdir",
        "chmod",
        "chown",
        "clock_getres",
        "clock_gettime",
        "clone",
        "close",
        "connect",
        "copy_file_range",
        "creat",
        "create_module",
        "dup",
        "dup2",
        "epoll_create",
        "epoll_ctl",
        "epoll_wait",
        "eventfd2",
        "execve",
        "execveat",
        "exit",
        "exit_group",
        "fadvise64",
        "fcntl",
        "fstat",
        "ftruncate",
        "futex",
        "get_kernel_syms",
        "getcwd",
        "getdents",
        "geteuid",
        "getpid",
        "getppid",
        "getrandom",
        "getrlimit",
        "getsockname",
        "getsockopt",
        "gettid",
        "gettimeofday",
        "getuid",
        "io_destroy",
```

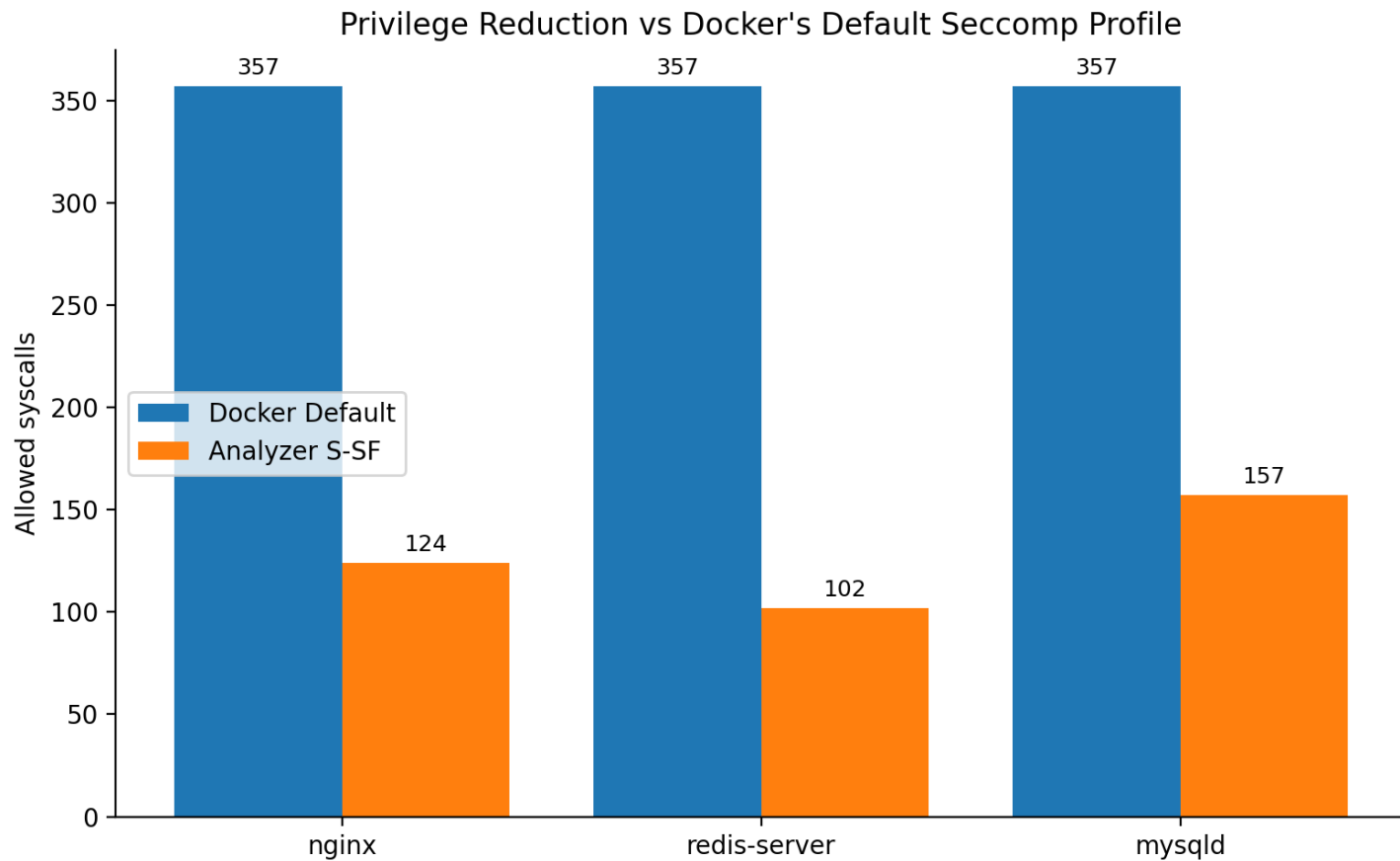
```
"io_destroy",
        "io_getevents",
        "io_setup",
        "ioctl",
        "kill",
        "listen",
        "lseek",
        "lstat",
        "madvise",
        "memfd_create",
        "mkdir",
        "mlock2",
        "mmap",
        "mprotect",
        "mremap",
        "munmap",
        "nanosleep",
        "newfstatat",
        "open",
        "openat",
        "pkey_alloc",
        "pkey_free",
        "pkey_mprotect",
        "prctl",
        "pread64",
        "preadv2",
        "pwrite64",
        "pwritev",
        "pwritev2",
        "query_module",
        "read",
        "readlink",
        "ready",
        "recvfrom",
        "recvmsg",
        "rename",
        "rmdir",
        "rt_sigaction",
        "rt_sigprocmask",
        "rt_sigreturn",
        "rt_sigsuspend",
        "sched_get_priority_max",
        "sched_get_priority_min",
        "sched_getparam",
        "sched_getscheduler",
        "sched_setaffinity",
        "sched_setscheduler",
        "sched_yield",
        "sendfile",
        "sendmsg",
        "sendto",
        "set_robust_list",
```

```
"set_robust_list",
        "setgid",
        "setgroups",
        "setitimer",
        "setpriority",
        "setrlimit",
        "setsid",
        "setsockopt",
        "setuid",
        "shutdown",
        "socket",
        "socketpair",
        "stat",
        "statfs",
        "sysinfo",
        "tgkill",
        "time",
        "umask",
        "umount2",
        "uname",
        "unlink",
        "uselib",
        "utimes",
        "vfork",
        "wait4",
        "write",
        "writev"
      ],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

Results



Results



PERFORMANCE METRICS

Seccomp Profiling Analysis

- **Attack Surface Reduction Rate:** Percentage of default Linux syscalls (out of ~300+) removed by the generated profile. Higher reduction indicates a smaller exploitable surface.
$$\text{Reduction Rate} = (1 - \text{Allowed Syscalls} / \text{Total Syscalls}) \times 100$$
- **False Positive Rate (Over-Restriction):** Percentage of syscalls the application legitimately needs but the profile incorrectly blocks, causing functional failures. Lower is better.
- **False Negative Rate (Under-Restriction):** Percentage of unnecessary/dangerous syscalls the profile mistakenly allows. Lower is better — this is what enables real exploits.
- **Segmentation Stability:** Variance in detected segmentation point T across repeated runs and across workload types, comparing fixed-threshold detection against adaptive change-point detection (Page-Hinkley/BOCPD). Lower variance indicates more consistent, workload-calibrated segmentation.
- **Coverage Ratio:** Ratio of syscalls captured by combined static + dynamic analysis ($S\text{-}SF \cup D\text{-}SF$) versus the true set of syscalls required across all application code paths.

PERFORMANCE METRICS

Anomaly Detection Analysis

- **Precision & Recall:** Proportion of flagged anomalies that are true attacks (Precision = $TP/(TP+FP)$), and proportion of actual attacks correctly flagged (Recall = $TP/(TP+FN)$).
- **F1-Score:** Harmonic mean of precision and recall, balancing false alarms against missed detections.
- **False Positive Rate (FPR):** Percentage of normal syscall sequences incorrectly flagged as anomalous — critical since high FPR causes legitimate workloads to be logged/blocked unnecessarily.
- **Detection Latency:** Time elapsed between an anomalous syscall sequence occurring and the Risk Scoring Engine issuing a Log/Alert or Block decision.
- **Runtime Overhead:** Additional CPU/memory cost introduced by continuous eBPF-based syscall capture and inference, measured against an unmonitored baseline container.

REFERENCES

- [1] K. Chen, H. Lu, Y. Yao, B. Fang, Y. Liu, and Z. Tian, "Enhancing Container Security Through Phase-Based System Call Filtering," *IEEE Transactions on Cloud Computing*, vol. 13, no. 3, pp. 983–994, Jul.–Sep. 2025.
- [2] H. Kang, E. Marin, M. You, D. Perino, S. Shin, and J. Kim, "BeaCon: Automatic Container Policy Generation Using Environment-Aware Dynamic Analysis," *Computers & Security*, vol. 162, art. no. 104789, 2026.
- [3] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, "DeSFAM: An Adaptive eBPF and AI-Driven Framework for Securing Cloud Containers in Real Time," *IEEE Access*, vol. 13, pp. 139203–139224, 2025.
- [4] R. Kim, J. Ryu, S. Kim, S. Lee, and S. Kim, "Detecting Cryptojacking Containers Using eBPF-Based Security Runtime and Machine Learning," *Electronics*, vol. 14, no. 6, p. 1208, 2025.
- [5] N. Zhou, H. Chen, Z. Chen, C. Li, and F. Li, "An Abnormal File Access Detection Model for Containers Based on eBPF Listening," *Mathematics*, vol. 14, no. 6, p. 991, 2026.

REFERENCES (Contd.)

- [6] J. Ryu, R. Kim, S. Lee, S. Kim, H. Choi, and S. Kim, "Hybrid Runtime Detection of Malicious Containers Using eBPF," *Computers, Materials & Continua*, vol. 86, no. 3, pp. 1–13, 2026.
- [7] S. Zehra, H. J. Syed, and U. Faseeha, "FedMon: Federated eBPF Monitoring for Distributed Anomaly Detection in Multi-Cluster Cloud Environments," in *Proc. 2025 20th Int. Conf. Emerging Technologies (ICET)*, Peshawar, Pakistan, 2025, pp. 1–7.
- [8] H. Taguchi, T. Hara, and S. Kasahara, "Unsupervised Real-Time In-Kernel Intrusion Detection System Using Autoencoders and eBPF," *IEICE Transactions on Communications*, vol. E109-B, no. 2, pp. 326–336, Feb. 2026.
- [9] J. Her, J. Kim, J. Kim, and S. Lee, "An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments," *IEEE Access*, vol. 13, pp. 155588–155604, 2025.
- [10] X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, "eBPF-Guard: A Detection Method for Container Escape via Multi-Level Monitoring and Enhanced Analysis Model," *Empirical Software Engineering*, vol. 31, art. no. 51, 2026.

THANK YOU